

NVIDIA

RTL PPA Optimization

ICLAD-DAC 2026 · GenAI Chip Hackathon

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

1 · The problem, and what actually scores

Given: 7 IPs (async_fifo, sha512, NVDLA, OpenTitan aes/ascon/kmac/prim), their testbenches, a Yosys+OpenSTA ASAP7 flow. **Rewrite RTL to improve PPA.**

Evaluation (per the organizers): functional correctness with the existing testbenches first; then PPA after Yosys, plus **LLM calls and token cost**.

Why this is hard (Alpha-RTL, ICCAD'24 lineage): on async_fifo, **every published LLM-rewriting method scored zero** — no compilable, correct rewrite. Unverified "optimizations" are noise; the synthesizer absorbs or breaks most local edits.

Our headline metric: baseline-normalized **Area-Delay-Product ratio** (clock period from each IP's own SDC), functional- and equivalence-gated, token-conscious.

2 · What we do — one picture

```
<svg viewBox="0 0 1160 470" style="width:100%;height:auto" xmlns="http://www.w3.org/2000/svg"> <defs>
<marker id="ar" viewBox="0 0 10 10" refX="9" refY="5" markerWidth="7" markerHeight="7" orient="auto-
start-reverse"><path d="M0 0L10 5L0 10z" fill="#1a3a6b"/></marker> <marker id="arg" viewBox="0 0 10 10"
refX="9" refY="5" markerWidth="7" markerHeight="7" orient="auto-start-reverse"><path d="M0 0L10 5L0
10z" fill="#1a7a2a"/></marker> <marker id="ara" viewBox="0 0 10 10" refX="9" refY="5" markerWidth="7"
markerHeight="7" orient="auto-start-reverse"><path d="M0 0L10 5L0 10z" fill="#b07a1a"/></marker> </defs>
<style> .t{font:600 14px sans-serif;fill:#1a3a6b}.s{font:12px sans-serif;fill:#333} .tiny{font:10.5px sans-
serif;fill:#555}.w{font:600 12.5px sans-serif;fill:#fff} .box{fill:#eef3fa;stroke:#1a3a6b;stroke-width:1.4;rx:8}
.gbox{fill:#e8f5e9;stroke:#1a7a2a;stroke-width:1.6;rx:8} .abox{fill:#fdf3e0;stroke:#b07a1a;stroke-
width:1.4;rx:8} .hdr{fill:#1a3a6b;rx:8} </style> <rect x="8" y="10" width="232" height="30" class="hdr"/> <text
x="124" y="30" text-anchor="middle" class="w">0 · DIAGNOSE — zero tokens</text> <rect x="16" y="52"
width="216" height="52" class="box"/> <text x="124" y="72" text-anchor="middle" class="t" font-
size="12.5">baseline synth + STA</text> <text x="124" y="90" text-anchor="middle" class="tiny">pristine
clone · Docker Yosys + OpenSTA → PPA0</text> <line x1="124" y1="104" x2="124" y2="122"
stroke="#1a3a6b" stroke-width="1.6" marker-end="url(#ar)"/> <rect x="16" y="124" width="216" height="88"
class="box"/> <text x="124" y="144" text-anchor="middle" class="t" font-size="12.5">3-axis diagnosis</text>
<text x="124" y="162" text-anchor="middle" class="tiny">timing: EL ATTEN=0 STA → worst path
```

3 · Principles

1. **Tools before tokens** — EDA tools diagnose *where* the problems are (0 model tokens); the model spends its tokens on findings, not file dumps.
2. **Verify everything** — five independent layers; accepted $\Rightarrow$ **formally equivalent AND measurably better**. Nothing is trusted, including our own guardrails.
3. **Measure, never estimate** — every candidate gets real synth+STA in an isolated workspace; every reject carries a typed, measured reason.
4. **Scope by construction** — the harness enforces which files a proposal may touch; out-of-scope edits are dropped by tooling, not by hoping the model complies.
5. **Honest reporting** — "no improvement" is a first-class, explained result; baselines are re-derived from pristine sources, never inherited.

4 · Zero-token PPA diagnosis

Run the EDA tools first, hand the model actionable findings.

- **Timing:** hierarchy-preserved synth+STA (`FLATTEN=0`) → worst-path cells attributed to **source files, in path order** → the campaign's file cursor
- **Area:** per-module cell counts — doubles as the leakage/power proxy
- **Structure classifier:** worst-path cell mix → `arith-carry-chain` / `mux-select` / `control-boolean-network` / ... → drives rule selection (slide 6)

Validated empirically on aes: naive per-module *depth* ranking and real hierarchy-preserved STA **disagree** — depth ≠ delay; we use the STA attribution.

Effect: aes prompt context **174k** → **~20k tokens/call**. Diagnosis cost: **0 tokens**.

5 · Staged coordinate descent

```
diagnose (0 tokens) → [worst file: k=6 strategies] → best win LOCKED (read-only)
                    → [next file: k=4]                → chain on best-so-far
                    → ... until the 20-proposal budget is spent (repairs off-budget)
```

Live sha512 campaign (fresh pool, 18 proposals, ~650k tokens):

Stage	File	Outcome	Accumulated ADP
1 (k=6)	sha512_core	ACCEPT arith-arch	0.938
2 (k=4)	sha512_w_mem	no win — best kept	0.938
3 (k=4)	sha512 (top)	ACCEPT on top of stage 1	0.920
4 (k=4)	h_constants	no win → plateau stop	0.920

Timing -97.3 ps → **+26.3 ps MET**; the stage-3 win compounds stage-1's — verified on disk.

`--focus` overrides the cursor for targeted campaigns (used for the unfenced aes S-box run).

6 · Context is a strategy — we ablated it

Same stage, same k=6, same strategies — only the **context** changes:

	A: full files (read-only)	B: target alone	C: interface stubs
Functionally correct	3/6	1/6	0/5
Accept	✓ 0.893	✓ 0.727	—
Tokens	329k	232k	236k

- Grounding buys **correctness**: the model needs the port/width contracts of the modules the target instantiates
- Leaner context saves tokens but starves behavioral detail (stubs: `pass=0` hangs)
- **Adopted: A** — full critical files as read-only grounding (cap 6), plus a **strict scope filter**: only the stage's file may change; everything else is dropped by tooling

7 · The 45-rule playbook — earned, matched, evolving

45 measured optimization rules, three provenances:

- **Literature**: distilled from our 20+ paper survey (prefix adders, Booth, strength reduction, operand isolation, clock gating, retiming notes, ...)
- **Our own experiments**: measured no-ops become AVOID rules — *"local restructuring (mux folding, operand swap) — ABC absorbs it; only global algebraic changes survive synthesis"*
- **Live reflector**: after each round the reflector distills outcomes into new rules and **votes** existing ones helpful/harmful (Beta posterior) — e.g. a live gate-fail became *"manual carry-save (3:2 compressor) insertion can break synthesis flows"*

Structure-matched retrieval: the diagnosis' classifier tag (e.g. `arith-carry-chain`) selects the *relevant* rules for each stage's prompt — the model gets 5 rules that match the circuit it is editing, not a lecture.

8 · Five-layer verification, then measurement

Layer	Catches
1 · lint	malformed RTL
2 · compile (iverilog)	elaboration errors → ≤2 repair attempts w/ stderr fed back
3 · testbench gate	functional breaks (differential gating vs pristine for IPs with pre-existing flow issues)
4 · synth + STA	real PPA; netlist-collapse guard (a "win" that lost half its cells is elaboration failure, not optimization)
5 · LEC (+async2sync) + dual-instance differential sim	logic-level & cycle-level equivalence

Accept = all layers pass AND measured strict improvement vs the best-so-far.

No-win stages trigger **PPA-ranked repair**: broken-but-improving candidates (measured!)

get one fix attempt each (top-2), *off* the proposal budget.

9 · Results scoreboard — complete seven-IP status

IP	Result	Assurance (per manifest)	Cost
sha512	ADP 0.727 , -97→+335 ps MET, area -0.4%	full 5-layer (gate PASS, LEC PROVEN, dualsim PASS)	8 calls / 232k
async_fifo	ADP 0.961 (micro-opt) — where published methods score 0	full 5-layer (LEC PROVEN)	offline
prim	ADP 0.605 , slack +181.5 ps, power -67%	equivalence+differential (LEC PROVEN + dualsim; gate skipped — pristine flow issue)	6 calls / 167k
aes (fenced / unfenced)	power -4.3% / -6% , ADP 1.00 — headroom sits in the S-box	differential-only (dualsim PASS; LEC inconclusive)	22 / 21 calls
kmac / ascon	offline candidates (Keccak- θ ; probes); live campaigns queued (Jul 19+)	5-layer capable	—
NVDLA	baseline + zero-config onboarding (323 sources); campaign on unlimited quota	—	—

Each artifact = **delta + manifest.json** with exact `verification_per_layer` + `assurance`.

10 · sha512 deep-dive: agent 0.727 beat our best hand-rewrite (0.787)

Metric	Baseline	Agent live (Jul 14)	Δ
WNS	−97.30 ps (violated)	+334.61 ps (MET)	+431.9 ps
Area	3984.2 μm^2	3967.6 μm^2	−0.4%
ADP ratio	1.000	0.727	−27.3%

Strategy `arith-arch` on `sha512_core` — a **single k=6 stage, 8 calls, ~232k tokens**.
yosys LEC: PROVEN. Differential sim: PASS.

Our best *hand-derived* rewrite (balanced adder trees, mod-2^w associativity) reached 0.787.
The agent's live rewrite **beats it by 6 points** — and earlier, its k-parallel proposals independently reinvented that same hand technique (caught by fingerprint dedup, zero wasted synthesis).

11 · prim deep-dive: "no improvement" flipped in 6 calls

Metric	Baseline	Agent (staged)	Δ
Setup slack	-208.95 ps	-27.46 ps	+181.5 ps
Area	70.17 μm^2	65.93 μm^2	-6.0%
Power	0.0165	0.00538	-67.4%
ADP ratio	1.000	0.6045	-39.6%

Earlier flat campaign: *no improvement* — *baseline is the submission*. The staged agent: **ADP 0.605 in 6 proposal calls / 167k tokens**, assurance = **equivalence+differential** (yosys LEC PROVEN + dual-instance differential sim; the pristine prim flow already fails compile/TB, so those two layers are pre-existing/skipped — not a full-5-layer claim).

The library-IP shape is why: prim ships **147 files but the scored design is one CRC32 module**. Diagnosis scopes the campaign to the file actually in the design — the third context regime (big IP · small IP · **library IP**) handled by the same mechanism.

12 · aes and the security fence — judgment, both numbers

OpenTitan aes ships a **DPA-masked S-box** (side-channel countermeasure). An agent can "win" PPA by silently swapping it for an unmasked LUT — every functional test still passes.

- **Fenced runs** (security preserved): power **-4.3%** (dualsim-verified), ADP 1.00 — and the agent's proposals kept reaching for the S-box: the timing headroom **is** the fence.
- The contest scores *tests-pass* + *PPA* — no security requirement. So the fence is **configurable** (`--fence` , default off per the rules). The targeted unfenced S-box campaign (`--focus`): power **-6.0%** (dualsim-verified), ADP still 1.00 — cycle-exact equivalence bounds the S-box axis to power wins; the delay headroom needs latency-changing rewrites → transaction-mode verification (our top Jul-19 item).

The agent quantifies the PPA cost of a security countermeasure — and enforces whichever posture the user chooses, by tooling rather than trust.

13 · Hidden testcases: drilled, not hoped for

Auto-discovery builds an IPSpec from repo conventions (env.sh, filelists, SDC, TB runners): fixtures match the hand registry 3/3; aes/kmac/prim/**NVDLA (323 sources)** onboard zero-config. Clock periods parse from each IP's own SDC.

Cold-start drill (`test_cold_start.py` , 6/6, re-run after every harness change): plant an unseen IP → discover → fresh baseline → propose → verify → measure → decide.

Robustness, live-fire tested: per-call resilient fan-out (one API failure never kills a campaign), HTTP timeouts, retry ladders on 429/5xx, failed calls don't consume the proposal budget, crash-protected repair paths. All ledgered: calls/tokens per round, raw responses archived.

14 · Next: what unlimited Vertex (Jul 19) unlocks

- **Model mixing** — strong model for creative proposals, cheap model for mechanical repairs/reflection. Our survey's scaling data (WNS 21/12/3%, SEC-pass 86/73/57% across model tiers) says capability buys *correct* rewrites — worth real tokens.
- **Transaction-mode differential sim** — legally accept latency-changing rewrites (unlocks the full S-box implementation axis on aes).
- **Richer interface stubs** (ports + latency contracts) — NVDLA-scale context bounding.
- **Per-round re-diagnosis** (the critical path moves as stages land) · growing stage-batch · decoding-config sweep · NVDLA + kmac/ascon campaigns.
- Agents remain updatable through **Jul 26**; live-testcase runbook (triage tree, budgets, panic modes) is in the repo.

15 · Summary — every claim, one command

Claim	Reproduce with
Loop offline e2e	<code>python3 -m ppa.controller --ip async_fifo --rounds 1 --k 1 --model stub</code>
Fresh baseline (Docker)	<code>python3 -m ppa.evaluate --ip <name path> --baseline</code>
5-layer verify	<code>python3 -m ppa.verify --ip sha512 --variant-dir ../exp2_sha512_balanced</code>
Hidden-testcase drill	<code>python3 test_cold_start.py</code> → 6/6
Model interface	<code>python3 test_model_iface.py</code> → 13/13
Staged campaign	<code>python3 -m ppa.controller --ip sha512 --rounds 8 --k 4 --k-first 6 --diagnose on --model vertex --max-calls 20 --emit-best ...</code>

Repo: github.com/chelsea85/chip-convergence-iclad26 (fresh-clone verified)

Companion deck: **Engineering Learnings** (attached) — what building this taught us.

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com